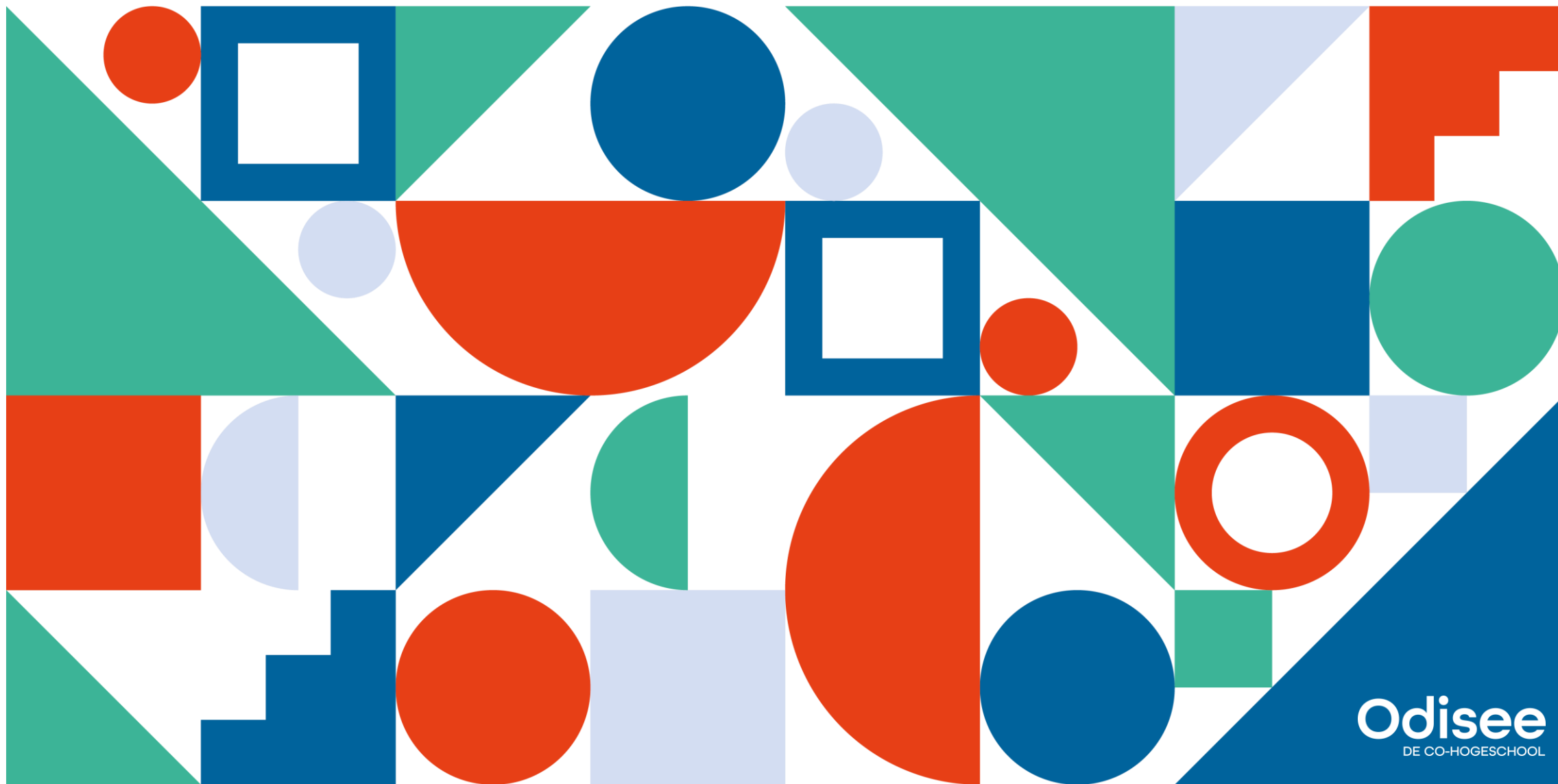




Odissee
DE CO-HOGESCHOOL



1.

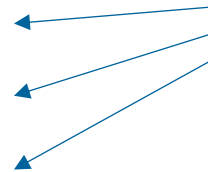
Joins

To join = samenvoegen

Relaties tussen tabellen?

- Een relatie (foreign key) beschrijft hoe rijen uit verschillende tabellen samengaan.
- De “employee werkt in departement” relatie bestaat uit de department_id kolommen van beide tabellen.

employee_id	first_name	last_name	department_id
103	Alexander	Hunold	60
104	Bruce	Ernst	60
107	Diana	Lorentz	60



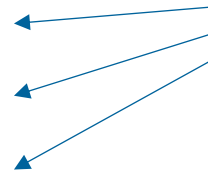
department_id	department_name
60	IT
80	Sales

Relaties tussen tabellen?

- Wat is de naam van de afdeling waarin Bruce Ernst werkt?
Hoe zou je dit oplossen zonder SQL?

employee_id	first_name	last_name	department_id
103	Alexander	Hunold	60
104	Bruce	Ernst	60
107	Diana	Lorentz	60

department_id	department_name
60	IT
80	Sales



Relaties tussen tabellen?

1. Vind Bruce Ernst

2. Noteer zijn department_id

employee_id	first_name	last_name	department_id
103	Alexander	Hunold	60
104	Bruce	Ernst	60
107	Diana	Lorentz	60

3. Zoek dit department_id

4. Noteer de naam

department_id	department_name
60	IT
80	Sales

Relaties tussen tabellen?

1. Vind Bruce Ernst
WHERE first_name =
'Bruce' AND last_name
= 'Ernst'

2. Noteer zijn
department_id
SELECT
department_id

3. Zoek dit department_id
WHERE
employees.department_id
=
departments.department_id

4. Noteer de naam
SELECT
department_name

employee_id	first_name	last_name	department_id
103	Alexander	Hunold	60
104	Bruce	Ernst	60
107	Diana	Lorentz	60

department_id	department_name
60	IT
80	Sales

Omzetten naar SQL

In de tabel employees

=> FROM employees

Zoeken we Bruce Ernst

=> WHERE first_name = 'Bruce' AND last_name = 'Ernst'

Daarna zoeken we in de DEPARTMENTS tabel

=> FROM departments

naar het department_id gevonden in de employees tabel

=> WHERE departments.department_id = employees.department_id

waarvan we de naam onthouden

=> SELECT department_name

Omzetten naar SQL

```
SELECT      department_name
FROM        employees, departments
WHERE       first_name = 'Bruce' AND last_name = 'Ernst'
           AND departments.department_id = employees.department_id
```



```
SELECT column_name(s)  
FROM table1, table2  
WHERE table1.column_name = table2.column_name;
```

Als er 2 kolommen dezelfde naam hebben (vb department_id) kan je ze onderscheiden door de tabelnaam ervoor te plaatsen.

- **Oefening jobomschrijving:**
Maak een lijst van alle werknemers. Geef de `employee_id`, voor- en familienaam en hun jobomschrijving welke ze uitvoeren.
- **Uitdaging Manager van de afdeling**
Maak een lijst van alle departementen met de namen van hun manager. Geef naam van het departement, familienaam en voornaam van de manager.

Kennen we nog aliassen?

- SQL aliases worden gebruikt om een **tabel**, of een **kolom** in een tabel, een **tijdelijke naam** te geven.
- Aliassen worden vaak gebruikt om tabel- en kolomnamen leesbaarder te maken
- Een alias bestaat alleen **voor de duur van een query**.

```
SELECT column_name(s)  
FROM table_name AS alias_name;
```

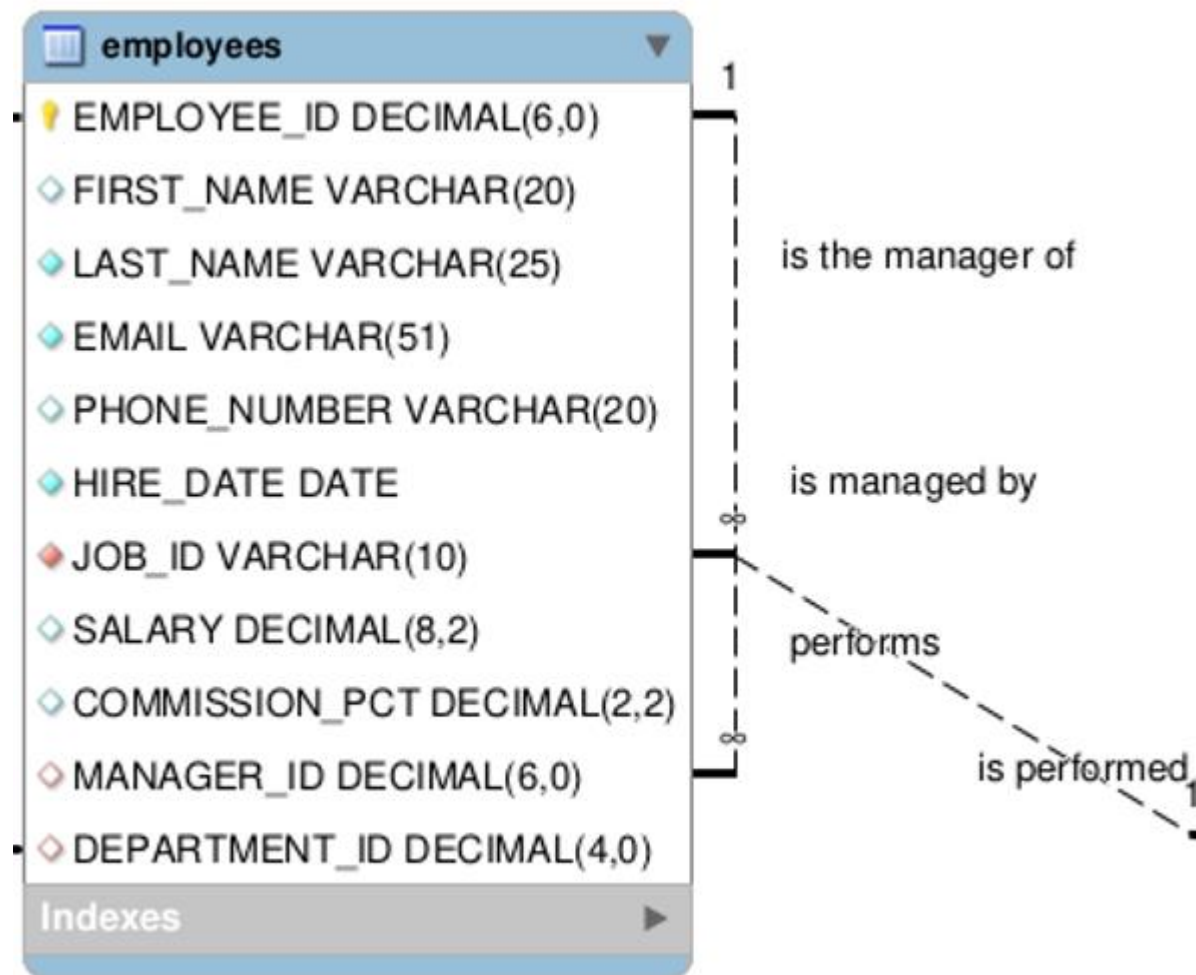
```
SELECT employee_id, first_name, last_name  
FROM employees AS Werknemers_met_commissie  
WHERE commission_pct is not NULL;
```

```
SELECT column_name(s)  
FROM table_name AS alias_name;
```

```
SELECT    department_name, last_name, first_name  
FROM      employees AS shipper , departments AS afdeling_shipping  
WHERE     department_name = 'shipping'  
AND       afdeling_shipping.department_id = shipper.department_id
```

- Geef de namen van de werknemers en de namen van hun manager

Werknemer Voornaam	Werknemer Familiennaam	Manager Voornaam	Manager Familiennaam
Neena	Kochhar	Steven	King
Lex	DeHaan	Steven	King



JOIN syntax

```
SELECT employee_id, e.job_id, job_title  
FROM employees AS e, jobs AS j  
WHERE e.job_id = j.job_id
```

is identiek aan:

```
SELECT employee_id, e.job_id, job_title  
FROM employees AS e JOIN jobs AS j  
    ON e.job_id = j.job_id
```

Pro: relaties scheiden van restricties
relaties definiëren samen met tabellen

JOIN JOIN JOIN JOIN JOIN JOIN JOIN JOIN

Een join van 2 tabellen resulteert in een nieuwe tabel. Deze kan met een andere tabel ge-JOIN-ed worden.

```
SELECT  employee_id, e.job_id, job_title
```

```
FROM    employees AS e JOIN jobs AS j
```

```
        ON e.job_id = j.job_id
```

```
        JOIN departments AS d
```

```
        ON e.department_id = d.department_id
```


JOIN JOIN JOIN JOIN JOIN JOIN JOIN JOIN

Een join van 2 tabellen resulteert in een nieuwe tabel. Deze kan met een andere tabel ge-JOIN-ed worden.

SELECT *employee_id, e.job_id, job_title*

FROM

employees AS e JOIN jobs AS j

ON *e.job_id = j.job_id*

JOIN departments AS d

ON *e.department_id = d.department_id*

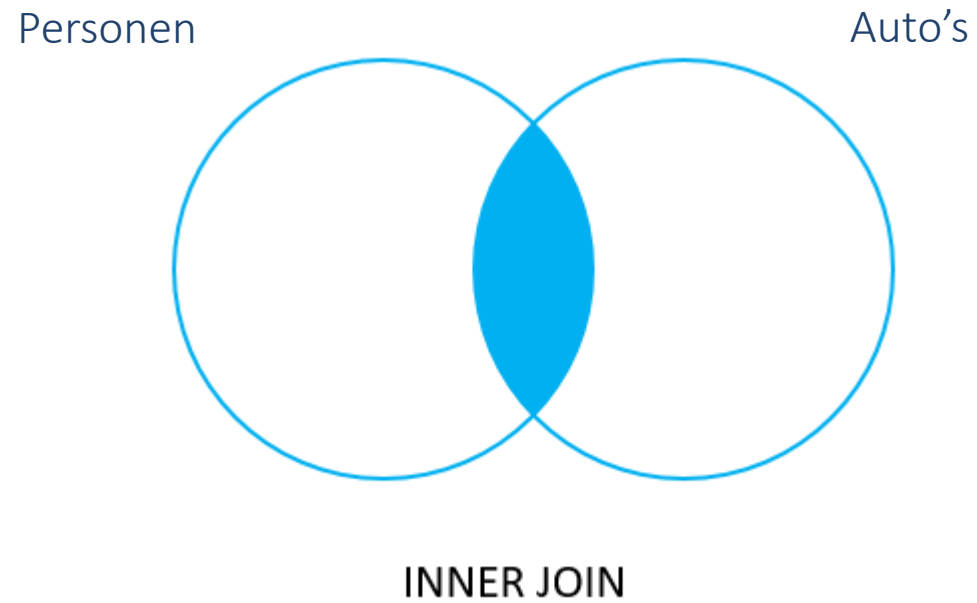
- **Oefening 3 tabellen met locatie**
Maak een lijst van alle werknemers. Geef de `employee_id`, `familienaam`, de `voornaam`, de naam van de afdeling waarin ze werken en het adres van de plaats waar dit departement zich bevindt. (zie tabel `locations`)
- **Oefening nog meer tabellen**
Hetzelfde als de vorige oefening. Je kan gerust de oplossing van je vorige oefening gebruiken als startpunt.
Naast de stad willen we ook weten in welk land dit ligt (land naam) en de naam van de regio.
Oh ja, en ook de jobtitel van de werknemer...

2. Outer Joins

Types Of Joins

- **(INNER) JOIN**: Geeft alle records terug die een overeenkomstige waarde hebben in beide tabellen
- **LEFT (OUTER) JOIN**: Geeft alle records van de linkse tabel, met de overeenkomstige records van de rechtse tabel
- **RIGHT (OUTER) JOIN**: Geeft alle records van de rechtse tabel, met de overeenkomstige records van de linkse tabel
- **FULL (OUTER) JOIN**: Geeft alle records van zowel de linkse als de rechtse tabel. FULL = LEFT + RIGHT

(INNER) JOIN



```
SELECT column_name(s)  
FROM table1  
INNER JOIN table2  
ON table1.column_name = table2.column_name;
```

(INNER) JOIN

ID	Naam
1	Tom
2	Bert
3	Marie
4	Linde

ID	Nummerplaat	EigenaarId
1	1-qwe-425	1
2	1-asd-546	2
3	1-jfg-458	2
4	1-abc-567	2
5	1-def-987	4
6	1-sdd-547	4
7	1-zxc-457	4
8	1-poi-524	5

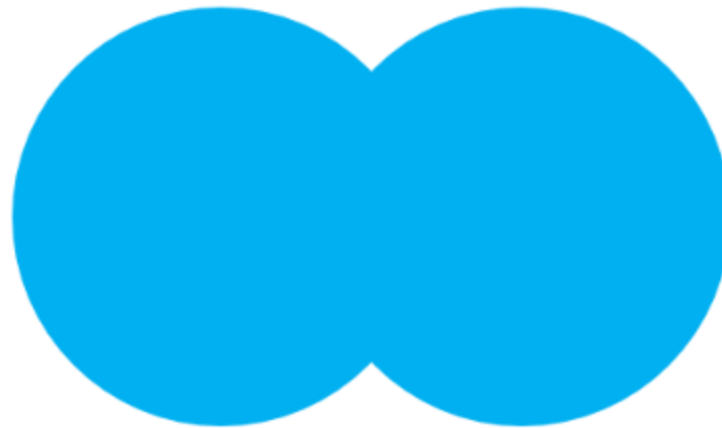


ID	Naam	ID	Nummerplaat	EigenaarId
1	Tom	1	1-qwe-425	1
2	Bert	2	1-asd-546	2
2	Bert	3	1-jfg-458	2
2	Bert	4	1-abc-567	2
4	Linde	5	1-def-987	4
4	Linde	6	1-sdd-547	4
4	Linde	7	1-zxc-457	4

FULL (OUTER) JOIN

Personen

Auto's



FULL OUTER JOIN


```
SELECT column_name(s)  
FROM table1 FULL OUTER JOIN table2  
      ON table1.column_name = table2.column_name  
[ ]WHERE condition;
```

FULL (OUTER) JOIN

ID	Naam
1	Tom
2	Bert
3	Marie
4	Linde

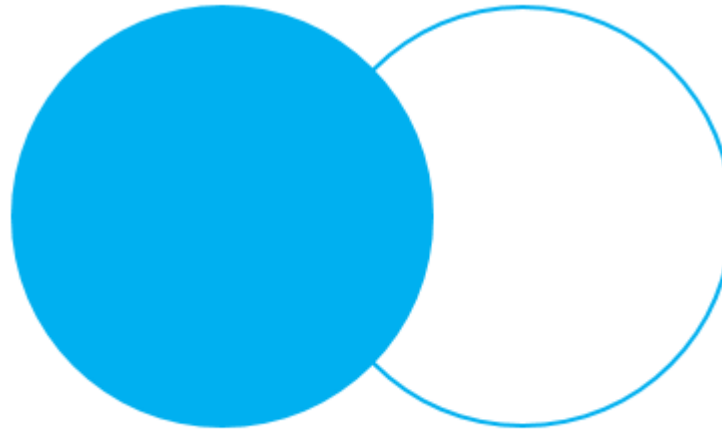
ID	Nummerplaat	EigenaarId
1	1-qwe-425	1
2	1-asd-546	2
3	1-jfg-458	2
4	1-abc-567	2
5	1-def-987	4
6	1-sdd-547	4
7	1-zxc-457	4
8	1-poi-524	NULL

ID	Naam	ID	Nummerplaat	EigenaarId
1	Tom	1	1-qwe-425	1
2	Bert	2	1-asd-546	2
2	Bert	3	1-jfg-458	2
2	Bert	4	1-abc-567	2
3	Marie	NULL	NULL	NULL
4	Linde	5	1-def-987	4
4	Linde	6	1-sdd-547	4
4	Linde	7	1-zxc-457	4
NULL	NULL	8	1-poi-524	5

LEFT (OUTER) JOIN

Personen

Auto's



LEFT OUTER JOIN

```
SELECT column_name(s)  
FROM table1 LEFT JOIN table2  
      ON table1.column_name = table2.column_name;
```

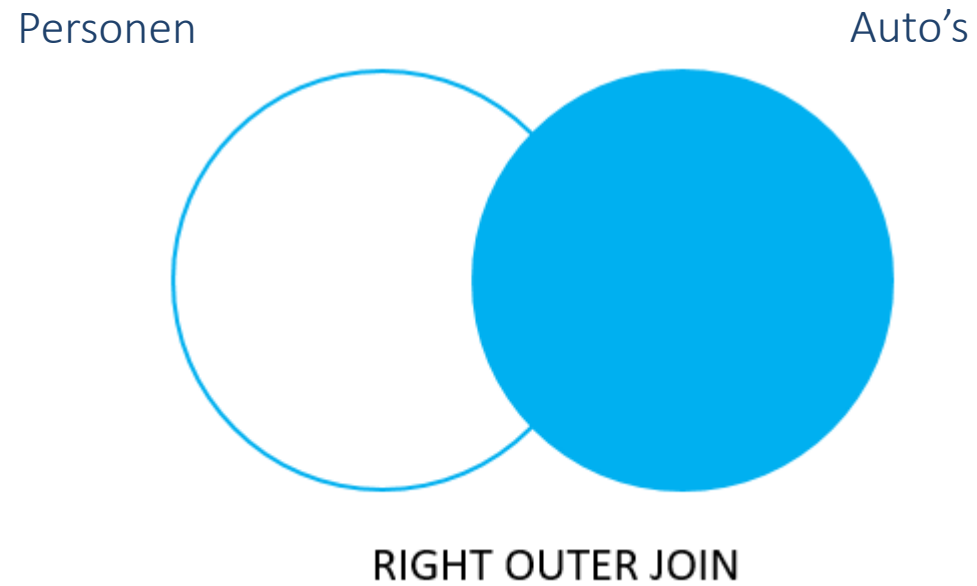
LEFT (OUTER) JOIN

ID	Naam
1	Tom
2	Bert
3	Marie
4	Linde

ID	Nummerplaat	EigenaarId
1	1-qwe-425	1
2	1-asd-546	2
3	1-jfg-458	2
4	1-abc-567	2
5	1-def-987	4
6	1-sdd-547	4
7	1-zxc-457	4
8	1-poi-524	NULL

ID	Naam	ID	Nummerplaat	EigenaarId
1	Tom	1	1-qwe-425	1
2	Bert	2	1-asd-546	2
2	Bert	3	1-jfg-458	2
2	Bert	4	1-abc-567	2
3	Marie	NULL	NULL	NULL
4	Linde	5	1-def-987	4
4	Linde	6	1-sdd-547	4
4	Linde	7	1-zxc-457	4

RIGHT (OUTER) JOIN



```
SELECT column_name(s)  
FROM table1 RIGHT JOIN table2  
      ON table1.column_name = table2.column_name;
```

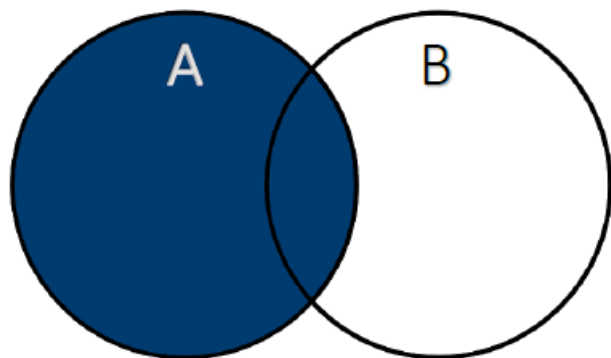
RIGHT (OUTER) JOIN

ID	Naam
1	Tom
2	Bert
3	Marie
4	Linde

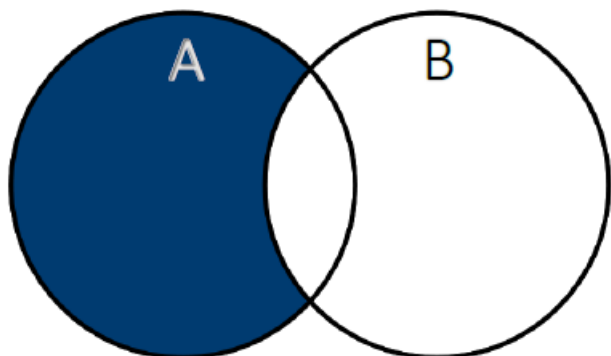
ID	Nummerplaat	EigenaarId
1	1-qwe-425	1
2	1-asd-546	2
3	1-jfg-458	2
4	1-abc-567	2
5	1-def-987	4
6	1-sdd-547	4
7	1-zxc-457	4
8	1-poi-524	NULL

ID	Naam	ID	Nummerplaat	EigenaarId
1	Tom	1	1-qwe-425	1
2	Bert	2	1-asd-546	2
2	Bert	3	1-jfg-458	2
2	Bert	4	1-abc-567	2
4	Linde	5	1-def-987	4
4	Linde	6	1-sdd-547	4
4	Linde	7	1-zxc-457	4
NULL	NULL	8	1-poi-524	NULL

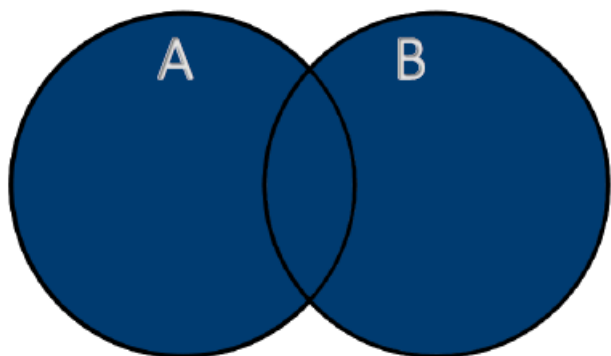
Joins Variations



LEFT INCLUSIVE

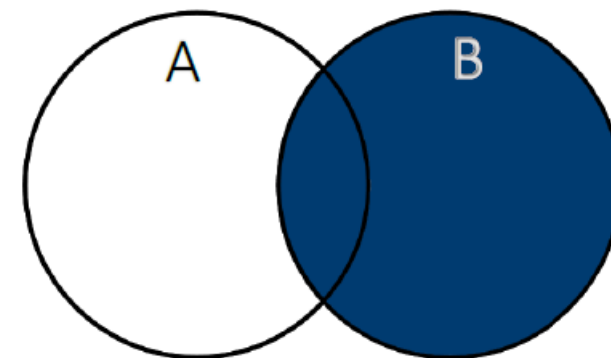


LEFT EXCLUSIVE

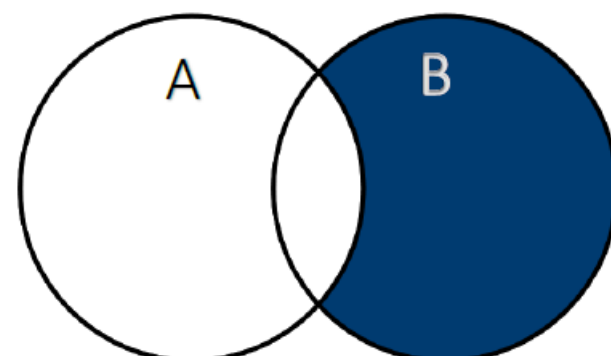


FULL OUTER INCLUSIVE

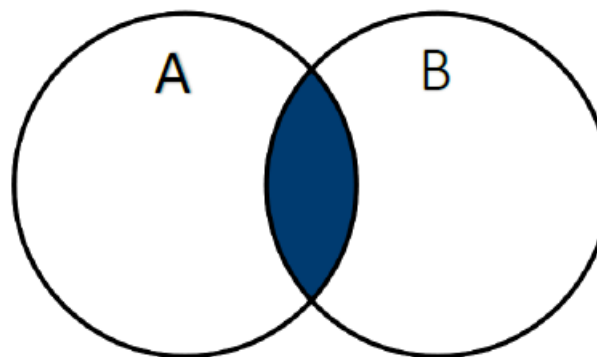
SQL JOINS	
LEFT INCLUSIVE SELECT [Select List] FROM TableA A LEFT OUTER JOIN TableB B ON A.Key= B.Key	RIGHT INCLUSIVE SELECT [Select List] FROM TableA A RIGHT OUTER JOIN TableB B ON A.Key= B.Key
LEFT EXCLUSIVE SELECT [Select List] FROM TableA A LEFT OUTER JOIN TableB B ON A.Key= B.Key WHERE B.Key IS NULL	RIGHT EXCLUSIVE SELECT [Select List] FROM TableA A LEFT OUTER JOIN TableB B ON A.Key= B.Key WHERE A.Key IS NULL
FULL OUTER INCLUSIVE SELECT [Select List] FROM TableA A FULL OUTER JOIN TableB B ON A.Key = B.Key	FULL OUTER EXCLUSIVE SELECT [Select List] FROM TableA A FULL OUTER JOIN TableB B ON A.Key = B.Key WHERE A.Key IS NULL OR B.Key IS NULL
INNER JOIN SELECT [Select List] FROM TableA A INNER JOIN TableB B ON A.Key = B.Key	



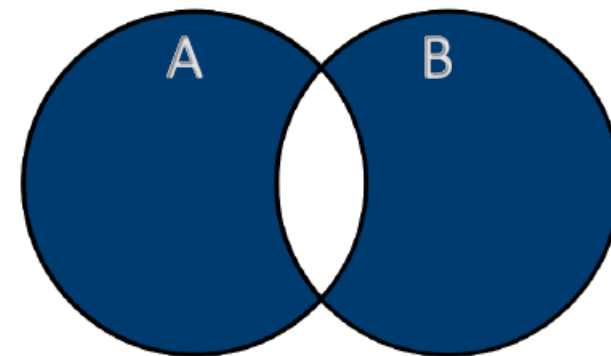
RIGHT INCLUSIVE



RIGHT EXCLUSIVE



INNER JOIN



FULL OUTER EXCLUSIVE

Wanneer in **2 tabellen dezelfde kolomnamen** gebruikt worden kunnen we in situaties komen waarbij SQL niet weet welke we bedoelen. De query is **ambiguous** voor de database.

Dit kunnen we oplossen door de kolomnaam te prefixen met de tabelnaam

```
SELECT ID  
FROM Car  
JOIN User ON ID = EigenaarID
```



```
SELECT User.ID  
FROM Car  
JOIN User ON User.ID = EigenaarID
```

WORLD DB

- Geef voor elk land het aantal steden terug. Ook landen die geen steden hebben.
- Geef voor elk continent (we willen alle continenten zien) het aantal talen dat er gesproken wordt.